

Experiment Report

Name: 李崇楷

Institution: NYCU AIIT

Course: Machine Learning

Student ID: 313834006

Code: <https://github.com/kailee0422/Machine-Learning.git>

Professor: Ching-Wen Ma

Date: 2024/09/21

Abstract

In Problem Set 1, we analyze logistic regression and Gaussian discriminant analysis (GDA) as two linear classification methods for binary classification. By computing the Hessian matrix and demonstrating its properties, we explore the convexity of logistic regression. We further examine GDA by analyzing its posterior probability and derive the maximum likelihood estimates for the parameters under certain assumptions. Finally, we provide computational solutions for training a logistic regression model using Newton's method and apply theoretical results to build a GDA classifier. These methods allow us to investigate the differences between discriminative and generative classifiers.

Object

Problem set1 (a): **Demonstrate the Convexity of the Logistic Regression Loss Function**

The goal of this problem is to calculate the Hessian matrix of the logistic regression loss function and prove that it is positive semi-definite, which implies that the loss function is convex. Convexity guarantees that the optimization process has a unique global minimum, ensuring efficient convergence of the training algorithm.

Problem set1 (b): **Implement Logistic Regression Using Newton's Method**

In this part, the objective is to implement Newton's Method for logistic regression. This involves iteratively updating the parameter vector θ by using both the gradient and the Hessian matrix of the loss function. The goal is to achieve convergence, i.e., update the parameters until the difference between consecutive iterations is sufficiently small.

Problem set1 (c): **Show that GDA Produces a Classifier with a Linear Decision Boundary**

The objective of this problem is to demonstrate that the posterior probability in Gaussian discriminant analysis (GDA) can be expressed in the same form as the logistic regression hypothesis. This shows that GDA results in a classifier with a linear decision boundary, although it is derived from a different probabilistic framework compared to logistic regression.

Problem set1 (d): **Make sense MLE for logistic regression**

In this part, the objective is to derive the maximum likelihood estimates (MLE) for the parameters in GDA, including ϕ, μ_0, μ_1 , and Σ . By maximizing the log-likelihood function of the data, we aim to confirm that the provided estimates for these parameters are correct and consistent with the GDA model assumptions.

Problem Methods and Result

Problem1:

In this problem, we cover two probabilistic linear classifiers we have covered in class so far. First, a discriminative linear classifier: logistic regression. Second, a generative linear classifier: Gaussian discriminant analysis (GDA). Both the algorithms find a linear decision boundary that separates the data into two classes, but make different assumptions. Our goal in this problem is to get a deeper understanding of the similarities and differences (and, strengths and weaknesses) of these two algorithms.

For this problem, we will consider two datasets, provided in the following files:

- i. `data/ds1_{train,valid}.csv`
- ii. `data/ds2_{train,valid}.csv`

Each file contains m examples, one example $(x^{(i)}, y^{(i)})$ per row. In particular, the i -th row contains columns $x_0^{(i)} \in \mathbb{R}$, $x_1^{(i)} \in \mathbb{R}$, and $y^{(i)} \in \{0, 1\}$. In the subproblems that follow, we will investigate using logistic regression and Gaussian discriminant analysis (GDA) to perform binary classification on these two datasets.

Problem set1 (a):

Problem:

(a) [10 points] In lecture we saw the average empirical loss for logistic regression:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})),$$

where $y^{(i)} \in \{0, 1\}$, $h_{\theta}(x) = g(\theta^T x)$ and $g(z) = 1/(1 + e^{-z})$.

Find the Hessian H of this function, and show that for any vector z , it holds true that

$$z^T H z \geq 0.$$

Hint: You may want to start by showing that $\sum_i \sum_j z_i x_i x_j z_j = (x^T z)^2 \geq 0$. Recall also that $g'(z) = g(z)(1 - g(z))$.

Remark: This is one of the standard ways of showing that the matrix H is positive semi-definite, written “ $H \succeq 0$.” This implies that J is convex, and has no local minima other than the global one. If you have some other way of showing $H \succeq 0$, you’re also welcome to use your method instead of the one above.

Solution and Result:

Step 1: Loss Function of Logistic Regression

The loss function for logistic regression can be defined as:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left(y^{(i)} \log(g(\theta^T x^{(i)})) + (1 - y^{(i)}) \log(1 - g(\theta^T x^{(i)})) \right)$$

where m is the number of samples, and $g(\theta^T x^{(i)})$ is the output of the sigmoid function, representing the predicted value for sample i :

$$g(\theta^T x^{(i)}) = \frac{1}{1 + e^{-\theta^T x^{(i)}}}$$

The goal is to derive the gradient

$\nabla J(\theta)$ and the Hessian matrix H , which is the second derivative of the loss function.

Step 2: Gradient Derivation of the Loss Function

We first compute the first derivative of the loss function with respect to the parameter θ_j :

$$\frac{\partial J(\theta)}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} g(\theta^T x^{(i)}) (1 - g(\theta^T x^{(i)})) x_j^{(i)} - (1 - y^{(i)}) g(\theta^T x^{(i)}) (1 - g(\theta^T x^{(i)})) x_j^{(i)} \right]$$

After simplification, we obtain:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (g(\theta^T x^{(i)}) - y^{(i)}) x_j^{(i)}$$

This is the gradient formula for logistic regression, and it can also be expressed in vector form:

$$\nabla J(\theta) = \frac{1}{m} X^T (g(X\theta) - Y)$$

where X is the feature matrix of all samples, $g(X\theta)$ is the predicted value vector for all samples, and Y is the vector of true labels.

Step 3: Hessian Matrix Derivation

Next, we compute the second derivative of the loss function with respect to θ_j and θ_k , which gives us the elements of the Hessian matrix H_{jk} :

$$H_{jk} = \frac{\partial^2 J(\theta)}{\partial \theta_j \partial \theta_k} = \frac{1}{m} \sum_{i=1}^m \left(\frac{\partial}{\partial \theta_k} (g(\theta^T x^{(i)}) - y^{(i)}) x_j^{(i)} \right)$$

Since $y(i)$ is not related to θ , we only need to consider the derivative of $g(\theta^T x^{(i)})$. Using the properties of the sigmoid function:

$$\frac{\partial}{\partial \theta_k} g(\theta^T x^{(i)}) = g(\theta^T x^{(i)}) (1 - g(\theta^T x^{(i)})) x_k^{(i)}$$

Therefore, the elements of the Hessian matrix can be expressed as:

$$H_{jk} = \frac{1}{m} \sum_{i=1}^m g(\theta^T x^{(i)}) (1 - g(\theta^T x^{(i)})) x_j^{(i)} x_k^{(i)}$$

This formula can be simplified in matrix form:

$$H = \frac{1}{m} X^T \text{diag}(g(X\theta) \circ (1 - g(X\theta))) X$$

Where $\text{diag}(g(X\theta) \circ (1 - g(X\theta)))$ is a diagonal matrix, with each diagonal element being $g(\theta^T x^{(i)})(1 - g(\theta^T x^{(i)}))$.

Step 4: Hessian Matrix Derivation

Finally, we verify the positive definiteness of the Hessian matrix. For any vector z , we consider the quadratic form $z^T H z$:

$$z^T H z = \frac{1}{m} \sum_{i=1}^m g(\theta^T x^{(i)}) (1 - g(\theta^T x^{(i)})) (z^T x^{(i)})^2$$

Due to the properties of the sigmoid function, we have $g(\theta^T x^{(i)})(1 - g(\theta^T x^{(i)})) \geq 0$, hence

$z^T H z \geq 0$. This indicates that the Hessian matrix is semi-positive definite.

Therefore, the loss function for logistic regression is a convex optimization problem, meaning its minimum is a global minimum.

Problem set1 (b):

Problem:

- (b) [5 points] **Coding problem.** Follow the instructions in `src/p01b_logreg.py` to train a logistic regression classifier using Newton's Method. Starting with $\theta = \vec{0}$, run Newton's Method until the updates to θ are small: Specifically, train until the first iteration k such that $\|\theta_k - \theta_{k-1}\|_1 < \epsilon$, where $\epsilon = 1 \times 10^{-5}$. Make sure to write your model's predictions to the file specified in the code.

Solution and Result:

Main function

```
# Create output directory if it doesn't exist
output_dir = os.path.dirname('output/')
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# Load training dataset
x_train, y_train = util.load_dataset(train_path, add_intercept=True)

# Initialize and train the logistic regression model using Newton's Method
model = LogisticRegression(eps=1e-5) # epsilon value for convergence
model.fit(x_train, y_train)

# Plot data and decision boundary
util.plot(x_train, y_train, model.theta, 'output/p01b_{}.png'.format(pred_path[-5]))

# Load evaluation dataset
x_eval, y_eval = util.load_dataset(eval_path, add_intercept=True)

# Predict and save predictions
y_pred = model.predict(x_eval)
np.savetxt(pred_path, y_pred > 0.5, fmt='%d')
```

Code Function:

- Initializes the LogisticRegression model and train it using the dataset loaded from `train_path` (here use `/content/ds1_train.csv`).
- Set a small epsilon (`1e-5`) as the convergence threshold.
- After training, it plots the data and the decision boundary using `util.plot` (Output: `/content/output/p01b_s.png`).
- Evaluate the model on the evaluation dataset loaded from `eval_path` (here use `/content/ds1_valid.csv`), predicts the outputs, and saves the results to `pred_path` (`/content/predictions.txt`).

Fit function

```
def fit(self, x, y):
    """Run Newton's Method to minimize J(theta) for logistic regression.

    Args:
        x: Training example inputs. Shape (m, n).
        y: Training example labels. Shape (m,).
    """
    # Initialize theta (weights)
    m, n = x.shape
    self.theta = np.zeros(n)

    while True:
        # Compute the hypothesis (sigmoid function)
        h_x = 1 / (1 + np.exp(-x.dot(self.theta)))

        # Compute the gradient of the cost function (first derivative)
        gradient = x.T.dot(h_x - y) / m

        # Compute the Hessian matrix (second derivative)
        H = (x.T * h_x * (1 - h_x)).dot(x) / m

        # Save the current value of theta
        theta_old = np.copy(self.theta)

        # Update theta using Newton's update rule
        self.theta -= np.linalg.inv(H).dot(gradient)

        # Check for convergence: Stop if the change in theta is less than epsilon
        if np.linalg.norm(self.theta - theta_old, ord=1) < self.eps:
            break
```

Code Function

- Initialize theta (the model weights) as a zero vector.
- Then, the method enters a loop where it:
 1. Computes the hypothesis using the sigmoid function.
 2. Calculates the gradient of the cost function (this is the first derivative of the cost function with respect to theta).
 3. Calculates the Hessian matrix (this is the second derivative, providing more information about the curvature of the cost function).
 4. Updates theta using Newton's update rule, where the change is based on the inverse of the Hessian matrix times the gradient.
 5. Checks for convergence, ensuring that the L1 norm of the change in theta is smaller than epsilon (eps)).

Predict function

```
def predict(self, x):  
    """Make a prediction given new inputs x.  
  
    Args:  
        x: Inputs of shape (m, n).  
  
    Returns:  
        Outputs of shape (m,).  
    """  
    return 1 / (1 + np.exp(-x.dot(self.theta)))
```

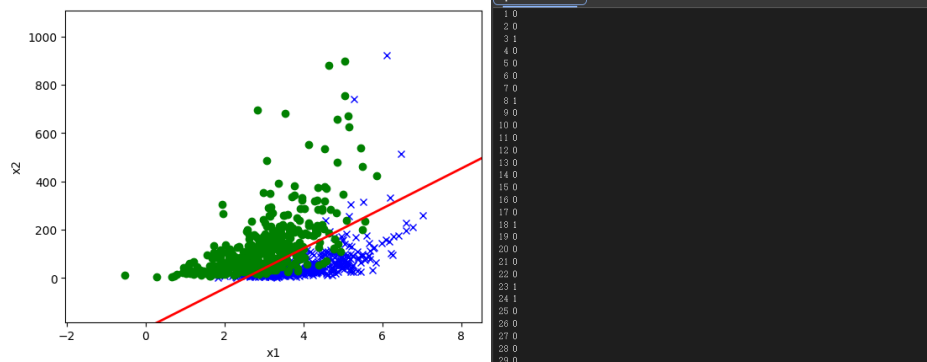
Code Function

- Takes new input x and returns predictions using the sigmoid function to classify the input as either 0 or 1 based on the learned theta.

Run Main Function

```
# Run the main function with provided dataset paths  
main('dsl_train.csv', 'dsl_valid.csv', 'predictions.txt')
```

Result



Summary

This program implements logistic regression using Newton's Method to optimize the parameters, **theta**. The main function loads the training dataset, trains the logistic regression model, plots the data along with the decision boundary, evaluates the model on a validation dataset, and saves the predictions. The LogisticRegression class contains two methods: **fit**, which trains the model by finding the optimal theta that minimizes the cost function using Newton's Method, and **predict**, which calculates the probability of class 1

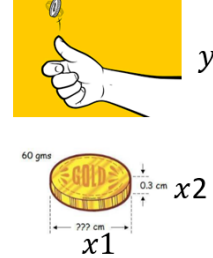
for the input data and applies a threshold to classify the result as either class 0 or 1. Newton's Method is used to minimize the cost function by iteratively updating theta, using both the gradient (first derivative) and Hessian matrix (second derivative) for precise updates, leading to faster convergence compared to gradient descent. The process stops when the change in theta across iterations is smaller than a defined threshold (epsilon = $1e-5$).

Problem set1 (c):

Problem:

- (c) [5 points] Recall that in GDA we model the joint distribution of (x, y) by the following equations:

$$\begin{aligned} p(y) &= \begin{cases} \phi & \text{if } y = 1 \\ 1 - \phi & \text{if } y = 0 \end{cases} \\ p(x|y=0) &= \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_0)^T \Sigma^{-1}(x - \mu_0)\right) \\ p(x|y=1) &= \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1)\right), \end{aligned}$$



where ϕ , μ_0 , μ_1 , and Σ are the parameters of our model.

Suppose we have already fit ϕ , μ_0 , μ_1 , and Σ , and now want to predict y given a new point x . To show that GDA results in a classifier that has a linear decision boundary, show the posterior distribution can be written as

$$p(y = 1 | x; \phi, \mu_0, \mu_1, \Sigma) = \frac{1}{1 + \exp(-(\theta^T x + \theta_0))},$$

where $\theta \in \mathbb{R}^n$ and $\theta_0 \in \mathbb{R}$ are appropriate functions of ϕ , Σ , μ_0 , and μ_1 .

Solution and Result:

Step 1: Express the posterior $p(y=1 | x)$

We start with the definition of the posterior probability using Bayes' theorem:

$$p(y = 1|x) = \frac{p(x|y = 1)p(y = 1)}{p(x|y = 1)p(y = 1) + p(x|y = 0)p(y = 0)}$$

Substitute in the equations for the conditional probabilities

$p(x | y=1)$, $p(x | y=0)$, and prior probabilities $p(y=1)=\phi$ and $p(y=0)=1-\phi$:

$$p(y = 1|x) = \frac{\exp\left(-\frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1)\right) \phi}{\exp\left(-\frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1)\right) \phi + \exp\left(-\frac{1}{2}(x - \mu_0)^T \Sigma^{-1}(x - \mu_0)\right) (1 - \phi)}$$

This is the key equation where we want to simplify the terms and express the posterior in a form that shows the linearity of the decision boundary.

Step 2: Simplify the numerator and denominator

We need to manipulate the terms to get the expression into a more compact form. The common exponential terms in the numerator and denominator hint at combining terms. To make the math easier, take the exponentials out of the denominator:

$$p(y = 1|x) = \frac{1}{1 + \frac{\exp\left(-\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0)\right)(1-\phi)}{\exp\left(-\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)\right)\phi}}$$

By simplifying the ratio of the exponentials, we get:

$$p(y = 1|x) = \frac{1}{1 + \exp\left(\frac{1}{2}(x - \mu_0)^T \Sigma^{-1}(x - \mu_0) - \frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1)\right) \cdot \frac{1-\phi}{\phi}}$$

Step 3: Expand and reorganize the terms

Next, expand the quadratic terms. The key observation is that:

$$\frac{1}{2}(x - \mu_0)^T \Sigma^{-1}(x - \mu_0) - \frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1)$$

can be rewritten as a linear function of x . This involves expanding both terms and combining like terms. After algebraic manipulation, we arrive at an expression of the form:

$$= (x^T \theta + \theta_0)$$

Where $\theta = \Sigma^{-1}(\mu_1 - \mu_0)$ and θ_0 is a constant term:

$$\theta_0 = \frac{1}{2}(\mu_0^T \Sigma^{-1} \mu_0 - \mu_1^T \Sigma^{-1} \mu_1) - \ln\left(\frac{1-\phi}{\phi}\right)$$

Step 4: Conclude the posterior as a logistic sigmoid

Now, substituting the linear expression back into the posterior equation, we get:

$$p(y = 1|x) = \frac{1}{1 + \exp(-(x^T \theta + \theta_0))}$$

This is the logistic sigmoid function, which shows that the decision boundary is linear in x , with the parameters θ and θ_0 determining the linearity.

Problem set1 (d):

Problem:

- (d) [7 points] For this part of the problem only, you may assume n (the dimension of x) is 1, so that $\Sigma = [\sigma^2]$ is just a real number, and likewise the determinant of Σ is given by $|\Sigma| = \sigma^2$. Given the dataset, we claim that the maximum likelihood estimates of the parameters are given by

$$\begin{aligned}\phi &= \frac{1}{m} \sum_{i=1}^m 1\{y^{(i)} = 1\} \\ \mu_0 &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\} x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} \\ \mu_1 &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\} x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} \\ \Sigma &= \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T\end{aligned}$$

The log-likelihood of the data is

$$\begin{aligned}\ell(\phi, \mu_0, \mu_1, \Sigma) &= \log \prod_{i=1}^m p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma) \\ &= \log \prod_{i=1}^m p(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma) p(y^{(i)}; \phi).\end{aligned}$$

By maximizing ℓ with respect to the four parameters, prove that the maximum likelihood estimates of ϕ , μ_0 , μ_1 , and Σ are indeed as given in the formulas above. (You may assume that there is at least one positive and one negative example, so that the denominators in the definitions of μ_0 and μ_1 above are non-zero.)

Solution and Result:

Step 1: Log-likelihood function

The total likelihood of the data given the parameters ϕ , μ_0 , μ_1 , and Σ is the product of the individual probabilities for each data point:

$$\ell(\phi, \mu_0, \mu_1, \Sigma) = \log \prod_{i=1}^m p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma)$$

Since $p(x^{(i)}, y^{(i)})$ can be written as $p(x^{(i)} | y^{(i)})p(y^{(i)})$, this expands to:

$$\ell(\phi, \mu_0, \mu_1, \Sigma) = \sum_{i=1}^m \log p(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma) + \sum_{i=1}^m \log p(y^{(i)}; \phi)$$

This splits the likelihood into the conditional likelihood of the data given the labels and the likelihood of the labels.

Step 2: Conditional likelihood term $p(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma)$

From the Gaussian assumption, $p(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma)$ is a multivariate Gaussian distribution. For the case $y^{(i)}=0$, the probability is:

$$p(x^{(i)} | y^{(i)} = 0; \mu_0, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x^{(i)} - \mu_0)^T \Sigma^{-1} (x^{(i)} - \mu_0) \right)$$

Similarly, for $y^{(i)}=1$, the probability is:

$$p(x^{(i)} | y^{(i)} = 1; \mu_1, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x^{(i)} - \mu_1)^T \Sigma^{-1} (x^{(i)} - \mu_1) \right)$$

The log of these terms simplifies the exponentials and eliminates the constant factors.

Step 3: Likelihood of the labels $p(y^{(i)}; \phi)$

The likelihood for the label $y^{(i)}$ is given by:

$$p(y^{(i)}; \phi) = \phi^{y^{(i)}} (1 - \phi)^{1-y^{(i)}}$$

Taking the log of this gives:

$$\log p(y^{(i)}; \phi) = y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi)$$

Step 4: Maximizing the log-likelihood

Now that we have the full log-likelihood expression, we maximize it with respect to each parameter by taking the derivative and setting it to zero:

- **Maximizing with respect to ϕ :**

$$\frac{\partial \ell}{\partial \phi} = \frac{1}{\phi} \sum_{i=1}^m 1(y^{(i)} = 1) - \frac{1}{1 - \phi} \sum_{i=1}^m 1(y^{(i)} = 0)$$

Setting this equal to zero leads to:

$$\phi = \frac{1}{m} \sum_{i=1}^m 1(y^{(i)} = 1)$$

which is the fraction of the samples where $y^{(i)}=1$.

- **Maximizing with respect to μ_0 and μ_1 :**

$$\frac{\partial \ell}{\partial \mu_0} = \Sigma^{-1} \sum_{i=1}^m 1(y^{(i)} = 0)(x^{(i)} - \mu_0)$$

Setting this to zero gives:

$$\mu_0 = \frac{\sum_{i=1}^m 1(y^{(i)} = 0)x^{(i)}}{\sum_{i=1}^m 1(y^{(i)} = 0)}$$

A similar procedure applies for μ_1 .

- Maximizing with respect to Σ :

$$\frac{\partial \ell}{\partial \Sigma} = \frac{1}{2} \Sigma^{-1} \left(\sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T - m \Sigma \right)$$

Setting this to zero gives the MLE for Σ :

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

Summary

The MLEs for ϕ , μ_0 , μ_1 , and Σ derived from maximizing the log-likelihood confirm the formulas given in the problem statement.

References

1. Maxim5. (2018). CS229 2018 Autumn Problem Sets and Solutions. GitHub. Retrieved from <https://github.com/maxim5/cs229-2018-autumn/tree/main/problem-sets-solutions/PS1>